Laboratorio 09 ReactJS

Desarrollo de Aplicaciones Empresariales

Estudiante: Pamela Meneses | Docente: Michael Montgomery Rosell | Fecha: 30 de octubre de 2026

Se desarrolló la cartelera de Órbita Cine con React y Vite dentro del proyecto acumulativo GLAB-S03-Meneses. La interfaz utiliza cinco componentes funcionales, datos de ocho películas, props y listas con claves estables.

El informe sigue los doce pasos de la guía. Los datos y los afiches son ficticios y se utilizan únicamente para la práctica.

Desarrollo y casos de prueba

1 Crear el proyecto con Vite

Estudiante: Pamela Meneses.

Se verificó Node.js v24.16.0 y npm 11.13.0. Se creó frontend con la plantilla React, se instalaron dependencias y se inició el servidor.

Resultado: Vite inicia en http://127.0.0.1:5173/.

Caso de prueba: Abrir el servidor: la interfaz debe responder.

```
node -v
npm create vite@latest frontend -- --template react --no-interactive
cd frontend
npm install
npm run dev
```

2 Limpiar la plantilla

Estudiante: Pamela Meneses.

Se reemplazó la demostración por index.html, src/main.jsx y src/App.jsx. Se retiraron los logos, el contador y los estilos de ejemplo.

Resultado: React monta App dentro de root y el documento utiliza el idioma español.

Caso de prueba: Recargar: no aparecen elementos de demostración.

Código: frontend/src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './styles/global.css'
import App from './App.jsx'
createRoot(document.getElementById('root')).render(<StrictMode><App /></StrictMode>)
```

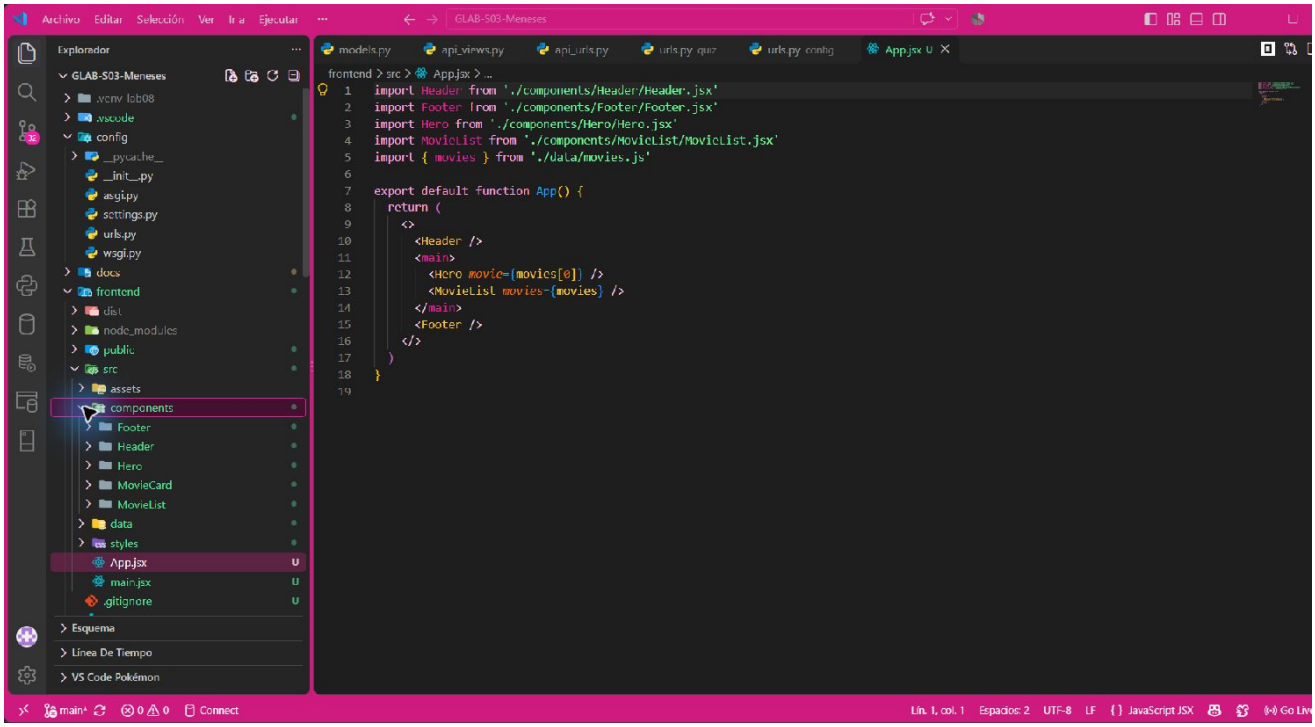
3 Organizar componentes datos y estilos

Estudiante: Pamela Meneses.

src/components tiene una carpeta por componente con JSX y CSS. src/data contiene movies.js y src/styles contiene global.css. Cada componente importa su CSS. La guía adjunta no detalla otra arquitectura CSS.

Resultado: Se separan datos, presentación y composición.

Caso de prueba: Revisar la captura de la estructura en VS Code.



Captura real del editor: App compone los componentes y el explorador muestra frontend/src.

4 Escribir los datos de la cartelera

Estudiante: Pamela Meneses.

movies.js exporta ocho películas ficticias con id, title, image, synopsis, classification, rating y showtimes. Los afiches son ilustraciones SVG locales creadas para esta práctica.

Resultado: Ocho registros completos y tres horarios por película.

Caso de prueba: Se observan ocho tarjetas y 24 horarios. Todas las imágenes cargan.

Código: frontend/src/data/movies.js

```
export const movies = [
{
  "id": 1,
  "title": "Beyond the Horizon",
  "image": "/images/horizon.svg",
  "synopsis": "Una astronauta recibe una seÃ±al imposible y viaja hacia los lÃ-mites del universo.",
  "classification": "PG-13",
  "rating": 8.8,
  "showtimes": [
    "15:00",
    "18:00",
    "21:00"
  ]
},
  // Seven additional movie objects are included in movies.js.
]
```

5 Crear Header

Estudiante: Pamela Meneses.

Header contiene el nombre del cine y navegación a Inicio, Cartelera y Contacto. Los enlaces apuntan a los identificadores de las secciones.

Resultado: Aparece ÓRBITA CINE con navegación.

Caso de prueba: Pulsar Cartelera: la dirección termina en #movies.

Código: frontend/src/components/Header/Header.jsx

```
import './Header.css'
export default function Header() {
  return (
    <header className="site-header">
      <a className="brand" href="#home"><span aria-hidden="true"> </span> ÓRBITA <small>CINE</small></a>
      <nav aria-label="Navegación principal">
        <a href="#home">Inicio</a><a href="#movies">Cartelera</a><a href="#contact">Contacto</a>
      </nav>
    </header>
  )
}
```



Recorte del resultado real en el navegador: header.

6 Crear Footer

Estudiante: Pamela Meneses.

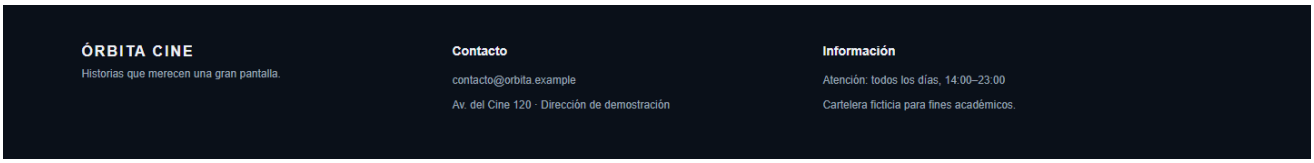
Footer contiene contacto e información. La dirección es de demostración y el correo utiliza el dominio reservado .example.

Resultado: Se muestran correo, dirección y horario de atención.

Caso de prueba: Pulsar Contacto: la dirección termina en #contact y se llega al pie.

Código: frontend/src/components/Footer/Footer.jsx

```
import './Footer.css'
export default function Footer() {
  return (
    <footer className="site-footer" id="contact">
      <div><strong>ÓRBITA CINE</strong><p>Historias que merecen una gran pantalla.</p></div>
      <div><h2>Contacto</h2><p>contacto@orbita.example</p><p>Av. del Cine 120 · Dirección de
    demostración</p></div>
      <div><h2>Información</h2><p>Atención: todos los días, 14:00–23:00</p><p>Cartelera ficticia para
    fines académicos.</p></div>
    </footer>
  )
}
```



Recorte del resultado real en el navegador: footer.

7 Crear Hero con props

Estudiante: Pamela Meneses.

App pasa movies[0] mediante la prop movie. Hero muestra el afiche, título, sinopsis, clasificación y valoración. El componente no importa los datos.

Resultado: Beyond the Horizon aparece como película destacada.

Caso de prueba: Comparar el banner con el primer objeto de movies.js.

Código: frontend/src/components/Hero/Hero.jsx

```
import './Hero.css'
export default function Hero({ movie }) {
  return (
    <section className="hero" id="home" aria-labelledby="featured-title">
      <img className="hero-image" src={movie.image} alt={movie.title} />
      <div className="hero-content">
        <p className="eyebrow">LA PELÍCULA DESTACADA</p>
        <p className="hero-meta">{movie.classification} <span>  {movie.rating.toFixed(1)} / 10</span></p>
        <h1 id="featured-title">{movie.title}</h1>
        <p className="hero-synopsis">{movie.synopsis}</p>
        <a className="primary-link" href="#movies">Explorar cartelera <span aria-
    hidden="true"> </span></a>
      </div>
      <p className="hero-caption">EN ÓRBITA, CADA HISTORIA ES UN VIAJE.</p>
    </section>
  )
}
```



Recorte del resultado real en el navegador: hero.

8 Crear MovieCard con props

Estudiante: Pamela Meneses.

MovieCard recibe movie y muestra su imagen, título, valoración y horarios. También presenta sinopsis y clasificación. Los horarios se recorren con map y usan cada horario como key, único dentro de la película.

Resultado: Una misma tarjeta presenta cualquier película recibida.

Caso de prueba: Cada tarjeta tiene datos propios y tres horarios.

Código: frontend/src/components/MovieCard/MovieCard.jsx

```
import './MovieCard.css'
export default function MovieCard({ movie }) {
  return (
    <article className="movie-card">
      <div className="poster-wrap">
        <img src={movie.image} alt={movie.title} loading="lazy" />
        <span className="movie-rating" aria-label={'Valoración: ' + movie.rating + ' de 10'}>
    {movie.rating.toFixed(1)}</span>
      </div>
      <div className="movie-title-row"><h3>{movie.title}</h3><span
    className="classification">{movie.classification}</span></div>
      <p className="movie-synopsis">{movie.synopsis}</p>
      <p className="showtimes-label">HORARIOS DE HOY</p>
      <ul className="showtimes" aria-label={'Horarios de ' + movie.title}>
        {movie.showtimes.map((showtime) => <li key={showtime}><time>{showtime}</time></li>)}
      </ul>
    </article>
  )
}
```



Recorte del resultado real en el navegador: card.

9 Crear MovieList con map y key

Estudiante: Pamela Meneses.

MovieList recibe movies desde App, recorre el arreglo con map y pasa cada objeto a MovieCard. La clave estable de cada tarjeta es movie.id.

Resultado: Se dibujan ocho tarjetas sin repetir manualmente el componente.

Caso de prueba: Se contaron 8 elementos article.movie-card y ninguna advertencia de key.

Código: frontend/src/components/MovieList/MovieList.jsx

```
import MovieCard from '../MovieCard/MovieCard.jsx'
import './MovieList.css'
export default function MovieList({ movies }) {
  return (
    <section className="movie-list" id="movies" aria-labelledby="movies-title">
      <div className="section-heading">
        <div><p className="eyebrow">ELIGE TU PRÓXIMA HISTORIA</p><h2 id="movies-title">En
    cartelera</h2></div>
        <p>{movies.length} películas en pantalla</p>
      </div>
      <div className="movie-grid">
        {movies.map((movie) => <MovieCard key={movie.id} movie={movie} />)}
      </div>
    </section>
  )
}
```

10 Componer la interfaz en App

Estudiante: Pamela Meneses.

App importa los datos y reúne Header, Hero, MovieList y Footer. Pasa la primera película a Hero y el arreglo completo a MovieList. Un fragmento agrupa los nodos raíz.

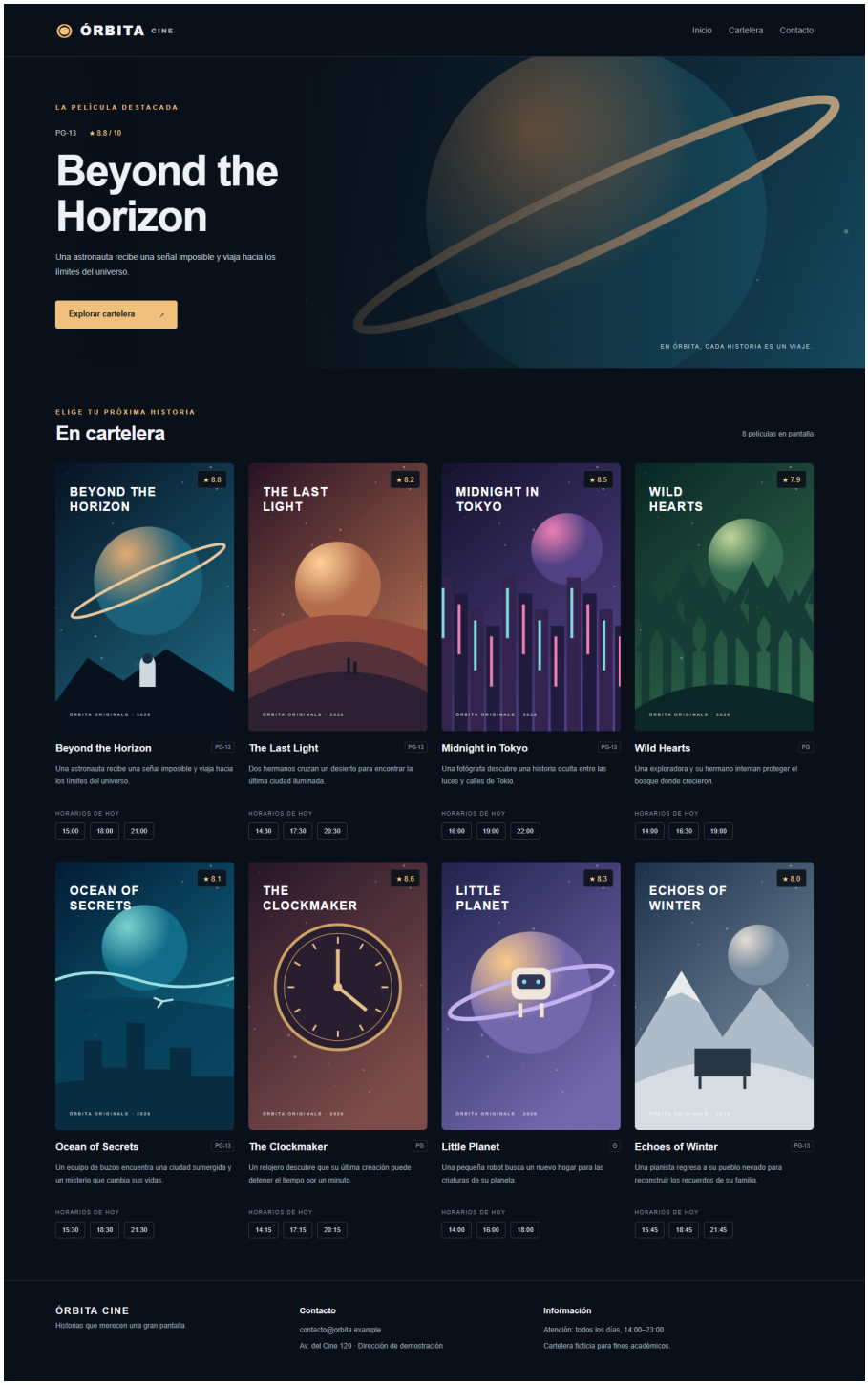
Resultado: Interfaz completa verificada en escritorio y móvil.

Caso de prueba: 8 tarjetas, 24 horarios, 9 imágenes cargadas, enlaces correctos, sin desbordamiento a 390 px y consola limpia.

Código: frontend/src/App.jsx

```
import Header from './components/Header/Header.jsx'
import Footer from './components/Footer/Footer.jsx'
import Hero from './components/Hero/Hero.jsx'
import MovieList from './components/MovieList/MovieList.jsx'
import { movies } from './data/movies.js'

export default function App() {
  return (
    <>
      <Header />
      <main>
        <Hero movie={movies[0]} />
        <MovieList movies={movies} />
      </main>
      <Footer />
    </>
  )
}
```



Cartelera completa: ocho películas y tres horarios por película.

11 Provocar y corregir los tres errores JSX

Estudiante: Pamela Meneses.

Se sustituyó temporalmente App.jsx por cada ejemplo defectuoso. Se capturaron mensajes reales y se restauró el archivo original. class y key generan advertencias; dos nodos raíz adyacentes impiden compilar.

Resultado: Los tres diagnósticos quedaron registrados en jsx-errors.json.

Caso de prueba: Se repitió la comprobación final: la consola quedó limpia.

Prueba 1: class instead of className

```
export default function App() { return <h1 class="title">JSX class test</h1> }

Invalid DOM property `%s`. Did you mean `%s`? class className
```

Corrección: Usar className en lugar de class.

Prueba 2: two root nodes

```
export default function App() { return (<h1>First root</h1><p>Second root</p>) }

[plugin:vite:oxc] Transform failed with 1 error:

[PARSE_ERROR] Adjacent JSX elements must be wrapped in an enclosing tag.
   ╭─[ src/App.jsx:1:60 ]
   │
 1 │ export default function App() { return (<h1>First root</h1><p>Second root</p>) }
   │                                                            ┬
   │                                                            ╰──
   │
   │ Help: Did you want a JSX fragment `<>...</>`?
───╯
```

Corrección: Agrupar los elementos con un fragmento <>...</>.

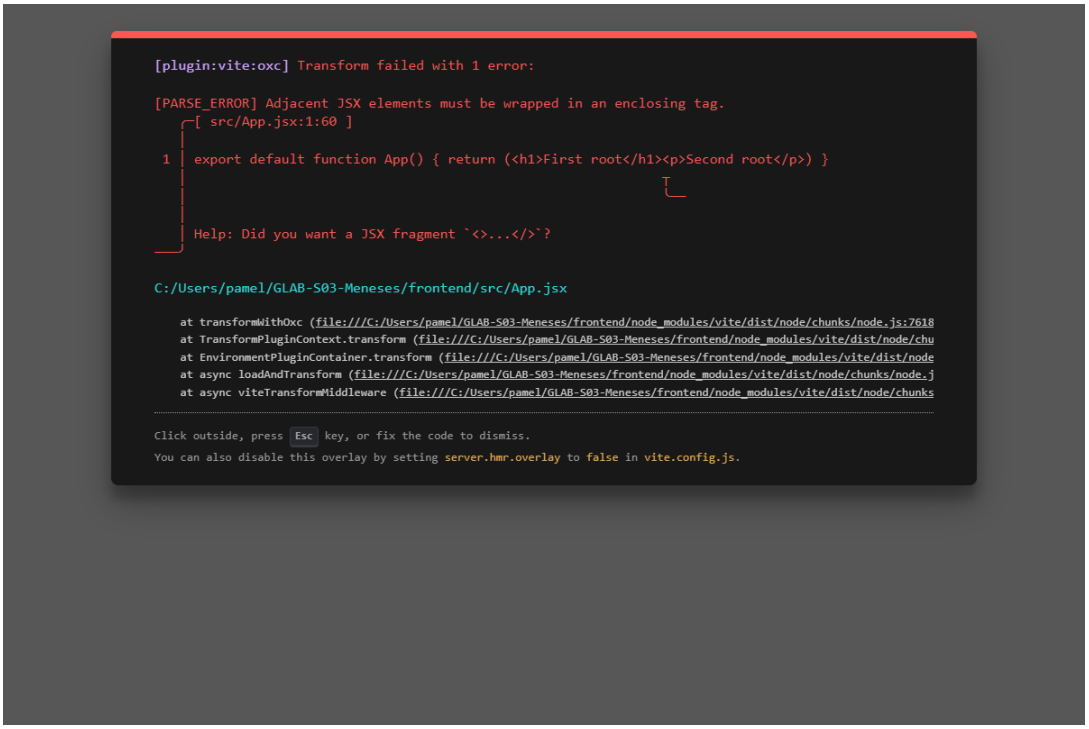
Prueba 3: missing key in map

```
export default function App() { return <ul>{['First movie', 'Second movie'].map((title) =>
    <li>{title}</li>)}</ul> }

Each child in a list should have a unique "key" prop.%s%s See https://react.dev/link/warning-keys for
    more information.

Check the render method of `App`.
```

Corrección: Agregar key={title} en este ejemplo; en la cartelera se usa key={movie.id}.



Captura real del error de compilación por nodos JSX adyacentes. Los mensajes de class y key se registraron desde la consola del navegador en jsx-errors.json; sus capturas de página muestran el resultado visual, no la consola.

12 Preparar repositorio y entregable

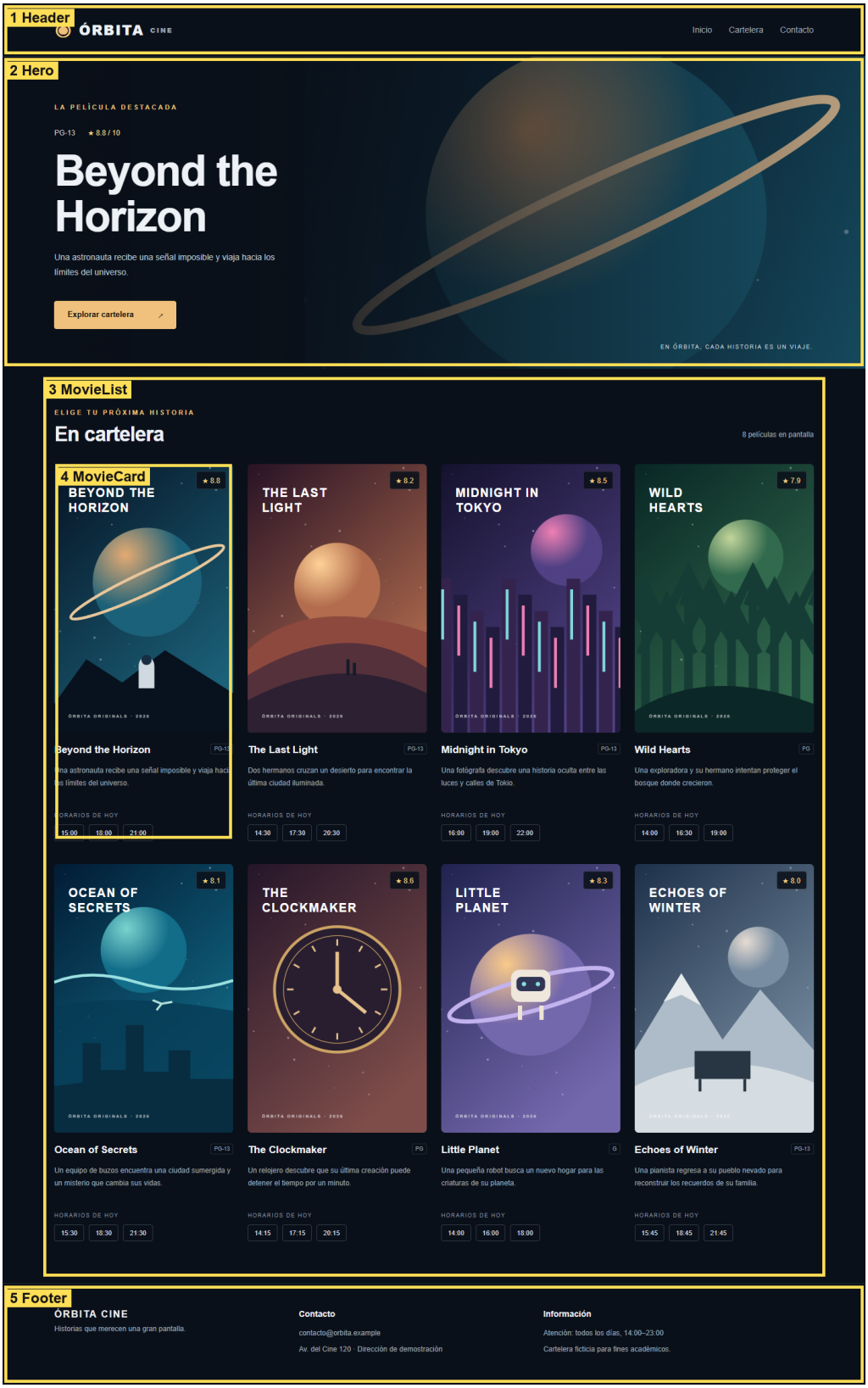
Estudiante: Pamela Meneses.

El frontend continúa el repositorio acumulativo. package.json y package-lock.json declaran dependencias; .gitignore excluye node_modules y dist. El entregable incluye capturas, código, explicación, casos de prueba y recorte en componentes.

Resultado: Archivos organizados en frontend/ y docs/lab09/.

Caso de prueba: Ejecutar npm ci, npm run build y npm run lint. Revisar integrantes y aportes reales antes de presentar en el campus.

Recorte de la interfaz en componentes



Captura real anotada: 1 Header, 2 Hero, 3 MovieList, 4 MovieCard, 5 Footer. App compone toda la interfaz.

Flujo de datos mediante props

```
App
 |-- Header
 |-- Hero          movie={movies[0]}
 |-- MovieList     movies={movies}
 |    |-- MovieCard key={movie.id} movie={movie}
 |-- Footer
```

Los hijos reciben los datos necesarios y no conocen su origen. Solo App importa el archivo de la cartelera. No se requiere consumo de API para los pasos de esta sesión.

Resumen de verificación

Comprobación	Resultado observado
Tarjetas	8
Horarios	24
Imágenes cargadas	9 de 9
Navegación	#movies y #contact correctos
Consola final	Sin errores ni advertencias
Vista móvil 390 px	Sin desbordamiento horizontal
Compilación y lint	Completados correctamente

Conclusiones

Los componentes funcionales dividen la interfaz por responsabilidad y permiten reutilizar MovieCard. Las props transmiten los datos desde App a Hero y MovieList, y desde MovieList a las tarjetas. map genera la cartelera y key identifica cada elemento. Los tres errores demostraron la sintaxis y las advertencias que deben corregirse antes de entregar.

Integrantes y aportes

Pamela Meneses figura como estudiante del entregable. La distribución de aportes entre los integrantes del equipo debe completarse con sus datos reales antes de presentar; no se han supuesto nombres ni responsabilidades.

Repositorio y presentación

Repositorio del equipo: https://github.com/pamelameneses-code/GLAB-S03-Meneses-D.E

Código: frontend/. Evidencias y documentación: docs/lab09/. El equipo debe presentar el entregable en el campus virtual dentro del plazo indicado por el docente.

Guía utilizada: GLAB-S09-MMONTGOMERY-2026-02.docx.